



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Form e HTTP POST

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- Form
- Richieste HTTP POST
- Modelli Pydantic



Aggiunta di un nuovo elemento

- Vogliamo estendere il sito del nostro finto negozio per fare in modo che si possano aggiungere nuovi prodotti tramite interfaccia web
- Abbiamo bisogno prima di tutto di una nuova pagina web, il cui template estenderà il "base.html". In quest'ultimo, possiamo già modificare il link nella barra di navigazione che indirizzerà alla nuova pagina web:

```
<a href="{{ url_for('add_product') }}">Add Product</a>
```

NB: stiamo già assumendo che la nuova funzione endpoint per mostrare questa pagina si chiamerà "add_product"! La definiamo quindi così:

```
@app.get("/add_product", response_class=HTMLResponse)
def add_product(request: Request):
    return templates.TemplateResponse(
        request=request, name="add_product.html"
    )
```

Pagina per l'aggiunta di un nuovo prodotto

- Creiamo un nuovo template "add_product.html" per l'aggiunta di un nuovo prodotto, contenente un form.

```
{% extends "base.html" %}
{% block content %}
    <form>
        <label for="name">Name:</label><br>
        <input type="text" name="name" id="name" required><br>
        <label for="price">Price:</label><br>
        <input type="number" min="0" name="price" id="price" required><br>
        <label for="location">Location:</label><br>
        <input type="text" name="location" id="location" required><br>
    <br><br>
        <input type="submit" value="Add">
        <input type="reset" value="Reset">
    </form>
{% endblock %}
```



Pagina per l'aggiunta di un nuovo prodotto

- Nel CSS aggiungiamo...

```
input[type=submit], input[type=reset] {  
    font-size: 16px;  
    background: #005599;  
    border: none;  
    color: white;  
    padding: 16px 32px;  
    cursor: pointer;  
}
```

Invio dati form

- Avviamo l'applicazione e andiamo nella pagina "Add Product".
- Cosa succede adesso se riempiamo i campi del form e clicchiamo "Add"? Viene effettuata una richiesta GET allo stesso URL, inserendo i parametri del form come coppie chiave-valore nei parametri dell'URL
- Proviamo quindi a modificare la funzione endpoint per raccogliere i dati inviati:

```
@app.get("/add_product", response_class=HTMLResponse)
def add_product(
    request: Request,
    name: str | None = None,
    price: float | None = None,
    location: str | None = None
):
    if name and price and location:
        product_list.append({"name": name, "price": price, "location": location})
    return templates.TemplateResponse(request=request, name="add_product.html")
```

Invio dati form

- Funziona, ma in questo caso non è l'ideale... ricevere i dati di un form nella stessa pagina è utile solo quando, ad esempio, dobbiamo modificare i contenuti della pagina stessa (con operazioni di filtraggio, ricerca, ordinamento, etc.).
- Nel nostro caso, dovendo aggiungere un nuovo dato, è meglio separare le responsabilità e usare una nuova funzione apposita:

```
@app.get("/insert_product_data")
def insert_product_data(
    name: str,
    price: float,
    location: str
):
    product_list.append(
        {"name": name, "price": price, "location": location}
    )
    return "Product added successfully"
```

Validazione dati form

- Modifichiamo anche il form con il link corretto:

```
<form action="/insert_product_data">
```

- Proviamo adesso a inviare di nuovo dei dati nel form, e verificare che il nuovo prodotto sia stato effettivamente aggiunto.
- ATTENZIONE: nonostante all'apparenza stiamo applicando una validazione avanzata tramite il form HTML per assicurarci che il prezzo sia maggiore di zero, questo non è sicuro!
- La validazione deve SEMPRE essere effettuata lato backend!
 - il file HTML può essere manipolato
 - l'endpoint può ricevere qualsiasi richiesta HTTP: proviamo con questa:
http://localhost:8000/insert_product_data?name=asdf&price=-1000&location=qwerty
- Serve quindi aggiungere una validazione avanzata anche lato backend...

Validazione avanzata dei dati

- Possiamo utilizzare il pattern di Python **Annotated** (importabile dalla libreria typing) per combinare. La sintassi è questa:

`Annotated[<type>, <metadata>, <metadata>, ...]`

- `<type>` è il tipo di dato atteso (notazione standard type hints)
- `<metadata>` sono uno o più parametri che possono avere varie funzionalità
- Nel nostro caso, useremo la funzione `Field` di Pydantic, che accetta diversi parametri per applicare dei vincoli sui dati. Esempi:

`Field(ge=1, le=5)` # `ge`: greater than or equal to; `le`: less than or equal to

`Field(min_length=3)` # `min_length`: minimum length of the string

- Documentazione completa: <https://docs.pydantic.dev/latest/concepts/fields/>



Validazione avanzata dei dati

- Modifichiamo così il nostro "main.py":

```
from typing import Annotated
from pydantic import Field

@app.get("/insert_product_data")
def insert_product_data(
    name: Annotated[str, Field(min_length=3, max_length=30)],
    price: Annotated[float, Field(gt=0)],
    location: Annotated[str, Field(min_length=3)]
):
```

- Adesso riproviamo a inviare la richiesta GET con dati non validi:
http://localhost:8000/insert_product_data?name=asdf&price=-1000&location=qwerty



HTTP POST

- Finora abbiamo sempre usato il metodo HTTP GET
 - le richieste (per convenzione) contengono parametri solo nell'URL (path e query parameter) e nell'header della richiesta HTTP
 - i dati però sono inviati in chiaro, e gli URL hanno una lunghezza limitata!
- Quando la richiesta HTTP necessita di contenere dati più sensibili e/o corposi, si usa il metodo POST che (per convenzione) può contenerli nel **body** della richiesta.
 - possiamo sempre e comunque usare anche i parametri nell'URL e nell'header HTTP
- FastAPI permette di definire gli endpoint per il metodo POST in maniera molto semplice:
 - usiamo il decoratore `@app.post("/path")`
 - definiamo una funzione di endpoint che prende come parametro la struttura dati che accettiamo, ed esegue internamente la logica necessaria

HTTP POST

- Modifichiamo il nostro form nell'HTML per fare in modo che invii i dati tramite richiesta POST...

```
<form action="/insert_product_data" method="post">
```

- ...e la funzione di endpoint perché risponda a richieste POST

```
@app.post("/insert_product_data")
```

- Cosa succede ora se proviamo a inviare dati attraverso il form? Non funziona!
 - la funzione endpoint si aspetta i parametri tramite URL query string...
 - ...il form ha inviato invece una richiesta POST, contenente i parametri nel body, con content-type application/x-www-form-urlencoded

Ricezione dati form

- Dobbiamo quindi ulteriormente modificare la funzione endpoint, ma basta poco:

```
from typing import Annotated
from pydantic import Field

@app.get("/insert_product_data")
def insert_product_data(
    name: Annotated[str, Form(), Field(min_length=3, max_length=30)],
    price: Annotated[float, Form(), Field(gt=0)],
    location: Annotated[str, Form(), Field(min_length=3)]
):
```

- Il campo Annotated può contenere diversi parametri e funzionalità, ne vedremo altre più avanti

Modelli Pydantic

- Abbiamo visto FastAPI usa Pydantic per la validazione dei dati, in modo da sollevare automaticamente eccezioni quando i dati in input/output non rispettano i tipi dichiarati
 - inoltre è possibile effettuare validazioni avanzate su ogni parametro
- Oltre che sui singoli parametri, Pydantic permette di lavorare su delle strutture dati chiamate **modelli**
 - ogni modello è definito tramite una classe (simile a una dataclass di Python) che eredita dalla class "BaseModel"
 - all'interno della classe/modello, possiamo definire i vari attributi, definendone il tipo, eventuale valore di default (solo se opzionali), e altri campi di validazione avanzata se necessari
- Nelle applicazioni implementate con FastAPI, questo è molto utile quando le funzioni endpoint ricevono numerosi parametri che possono essere raggruppati insieme e riutilizzati in altre funzioni

Modelli Pydantic

- Definiamo un modello per rappresentare un prodotto

```
from pydantic import BaseModel, Field
```

```
class Product(BaseModel):  
    name: str  
    price: float  
    location: str
```

- Per creare un oggetto Product, basta istanziare la sua classe passando gli argomenti
 - è già in questo momento che avviene la validazione dei parametri...

```
product = Product(name="Notebook", price=50.00, location="Cagliari")  
print(product.name)
```



Modelli Pydantic

- Aggiungiamo anche la validazione avanzata...

```
class Product(BaseModel):  
    name: Annotated[str, Field(min_length=3, max_length=30)]  
    price: Annotated[float, Field(gt=0)]  
    location: Annotated[str, Field(min_length=3)]
```

- È possibile convertire un modello da/a JSON/dizionario utilizzando delle semplici funzioni:

```
product = Product.model_validate( # model_validate_json per JSON  
    {"name": "Notebook", "price": 50.00, "location": "Cagliari"}  
)  
product.model_dump() # model_dump_json per file JSON
```


Form con modello Pydantic

- Adesso possiamo modificare la funzione endpoint che riceve i dati del form, in modo che usi il modello Pydantic

```
product_list = [  
    Product(name="Notebook", price=50.00, location="Cagliari")  
]
```

```
@app.post("/insert_product_data")  
def insert_product_data(  
    product: Annotated[Product, Form()]  
):  
    product_list.append(product)  
    return "Product added successfully"
```

HTTP POST con JSON

- I modelli Pydantic sono molto utili anche quando si implementano endpoint che accettano richieste POST con JSON nel body della richiesta HTTP
- In questo caso è ancora più semplice, proviamo a fare una funzione che riceve un prodotto:

```
@app.post("/insert_product_json")  
def insert_product_json(product: Product):  
    product_list.append(product)  
    return "Product added successfully"
```

- Potremmo testarla usando la libreria request, ma ne approfittiamo per introdurre un'altra funzionalità di FastAPI:
<http://localhost:8000/docs>